Lecture 15: Transformers

Instructor: Stephanie Schoch

CSCI 680: Natural Language Processing
Department of Computer Science, College of William & Mary
October 16, 2026

WILLIAM & MARY

Many slides are adapted from Jurafsky & Martin (3rd ed.) & Chris Manning.

Lecture Outline

- Recap: Self-Attention
- Multiheaded Attention
- Transformers

Recap: Self-Attention

Attention for Passing Information in Networks

Abstractly: Attention is a way to pass information from a sequence (x) to a neural network input. (h_t)

- This is also *exactly* what RNNs are used for – to pass information!
- **Can we just get rid of the RNN entirely?** Maybe attention is just a better way to pass information!

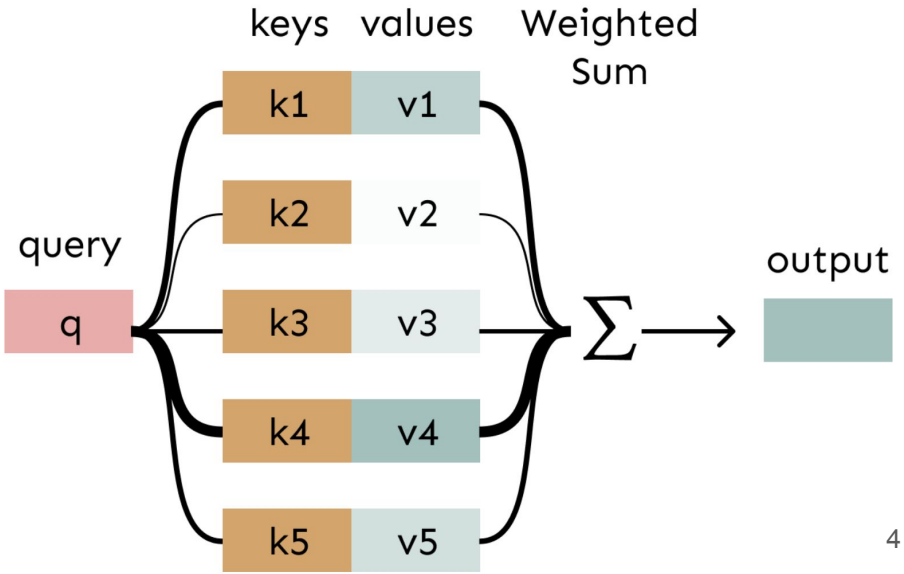
Cross attention: paying attention to the input x to generate y_t

Self attention: to generate y_t, we need to pay attention to $y_{<t}$

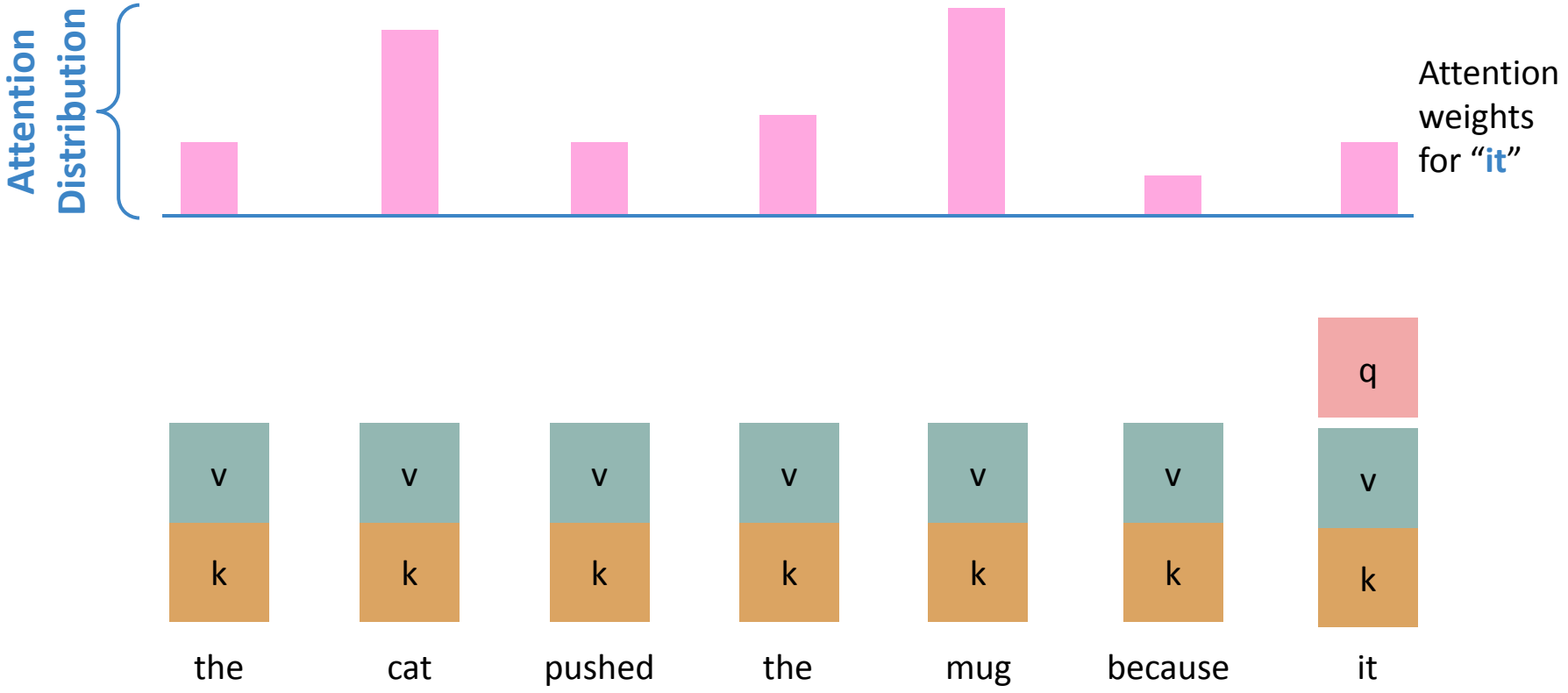
Self-attention allows a network to directly extract & use information from arbitrarily large contexts without the need to pass it through intermediate recurrent connections as in RNNs

Attention is weighted averaging

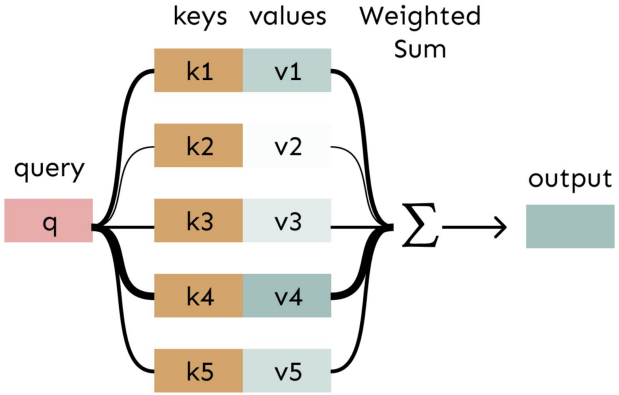
In **attention**, the query matches all keys *softly*, to a weight between 0 and 1. The keys' values are multiplied by the weights and summed.



Self-Attention: Hypothetical Example



Self-Attention



Let $\mathbf{w}_{1:N}$ be a sequence of words in vocabulary V

For each w_i, let $\mathbf{x}_i = \mathbf{E}_{w_i,:} \in \mathbb{R}^{1\times d}$, where

$\mathbf{E} \in \mathbb{R}^{|\mathcal{V}|\times d}$ is an embedding matrix.

1. Transform each word embedding with weight matrices $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V$

$$\mathbf{W}^Q \in \mathbb{R}^{d\times d_k} \qquad \mathbf{W}^K \in \mathbb{R}^{d\times d_k} \qquad \mathbf{W}^V \in \mathbb{R}^{d\times d_v}$$

$$\mathbf{q}_i = \mathbf{x}_i\mathbf{W}^Q \in \mathbb{R}^{1\times d_k} \qquad \mathbf{k}_i = \mathbf{x}_i\mathbf{W}^K \in \mathbb{R}^{1\times d_k} \qquad \mathbf{v}_i = \mathbf{x}_i\mathbf{W}^V \in \mathbb{R}^{1\times d_v}$$

2. Compute similarities between keys and queries; normalize with softmax

$$e_{ij} = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} \qquad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'=1}^{N}\exp(e_{ij'})}$$

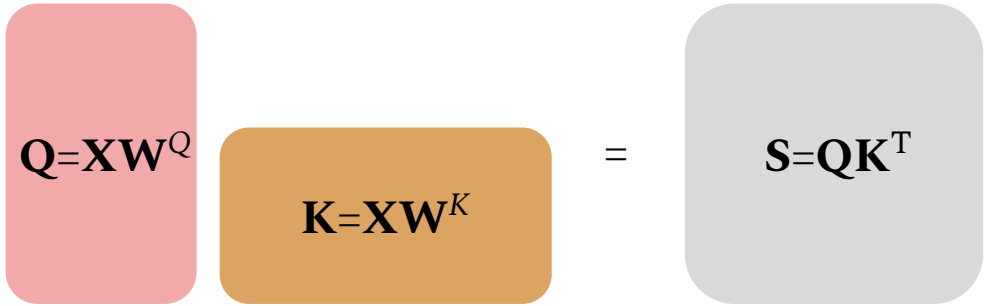
3. Compute output for each word as weighted sum of values: $\mathbf{o}_i = \sum_{j=1}^{N}\alpha_{ij}\mathbf{v}_j \in \mathbb{R}^{1\times d_v}$

Self-Attention: Matrix Multiplications

- Key-query-value attention is typically computed as matrices.
- Let $\mathbf{X} = \begin{bmatrix} \mathbf{x}_1 \\ \vdots \\ \mathbf{x}_N \end{bmatrix} \in \mathbb{R}^{N \times d}$ be the concatenation of input vectors
- Query, Key, Value matrices: $\mathbf{Q} = \mathbf{X}\mathbf{W}^Q \in \mathbb{R}^{N \times d_k}$, $\mathbf{K} = \mathbf{X}\mathbf{W}^K \in \mathbb{R}^{N \times d_k}$, $\mathbf{V} = \mathbf{X}\mathbf{W}^V \in \mathbb{R}^{N \times d_v}$
- Score matrix: $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}$
- Attention matrix: $\mathbf{A} = \mathrm{softmax}_{\mathrm{row}}\left(\frac{\mathbf{S}}{\sqrt{d_k}}\right)$
- Output matrix: $\mathbf{O} = \mathbf{A}\mathbf{V} \in \mathbb{R}^{N \times d_v}$

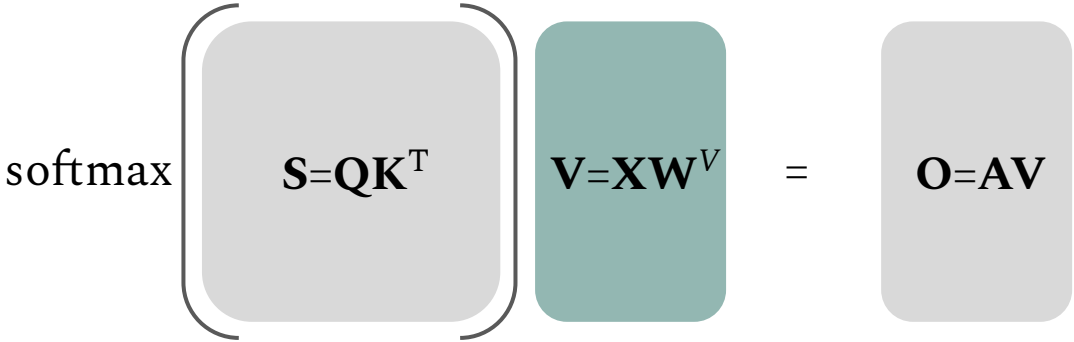
Self-Attention: Matrix Multiplications

First, take the query-key dot products in one matrix multiplication

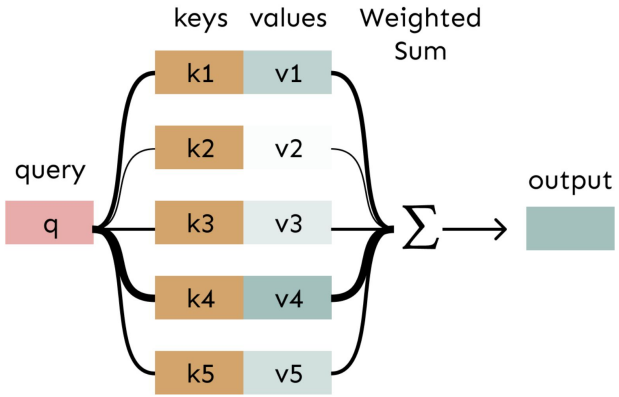


All pairs of attention scores!

Next, softmax, and compute the weighted average with the values (with another matrix multiplication)



Why Self-Attention?



- Self-attention allows a network to directly extract and use information from arbitrarily large contexts without the need to pass it through intermediate recurrent connections as in RNNs
- Used often with feedforward networks!

Transformers are Self-Attention Networks

- Self-Attention is the key innovation behind Transformers!
- Transformers map sequences of input vectors $\mathbf{x}_{1:N}$ to sequences of output vectors $\mathbf{y}_{1:N}$ of the same length.
- Made up of stacks of Transformer blocks
  - each of which is a multilayer network made by combining
    - simple linear layers,
    - feedforward networks, and
    - self-attention layers
- No recurrent connections!



Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

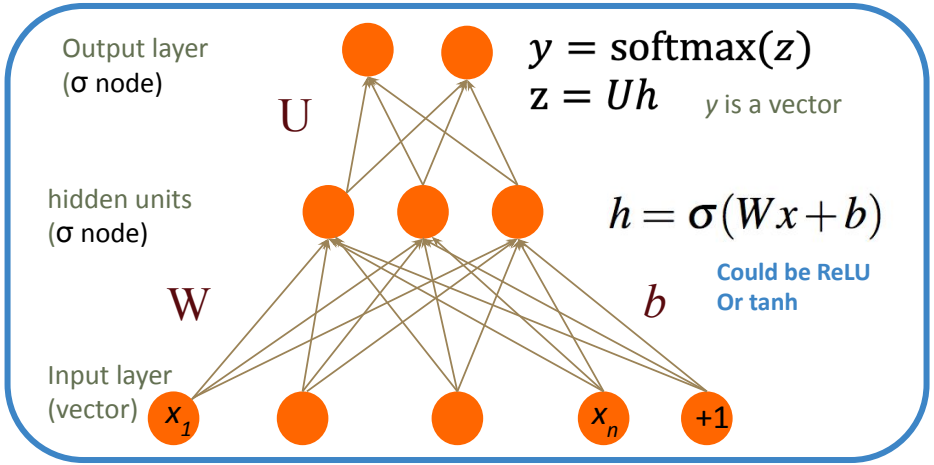
Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.

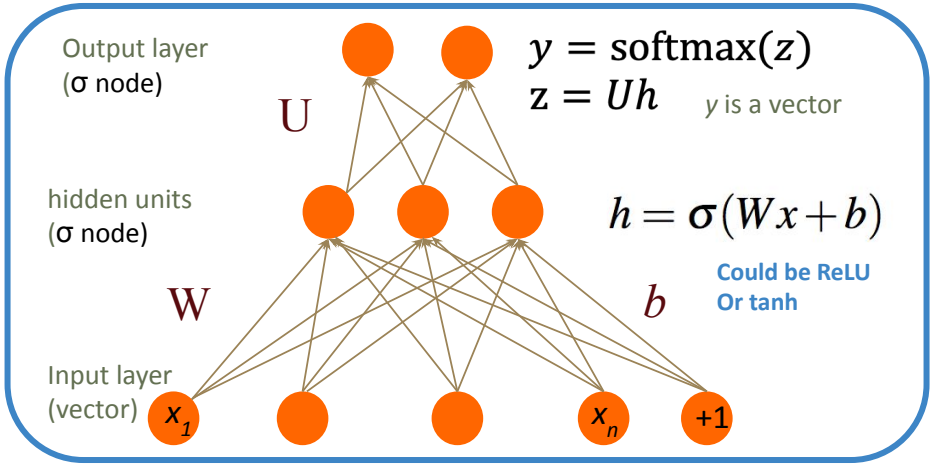
Adding Nonlinearities in Self-Attention

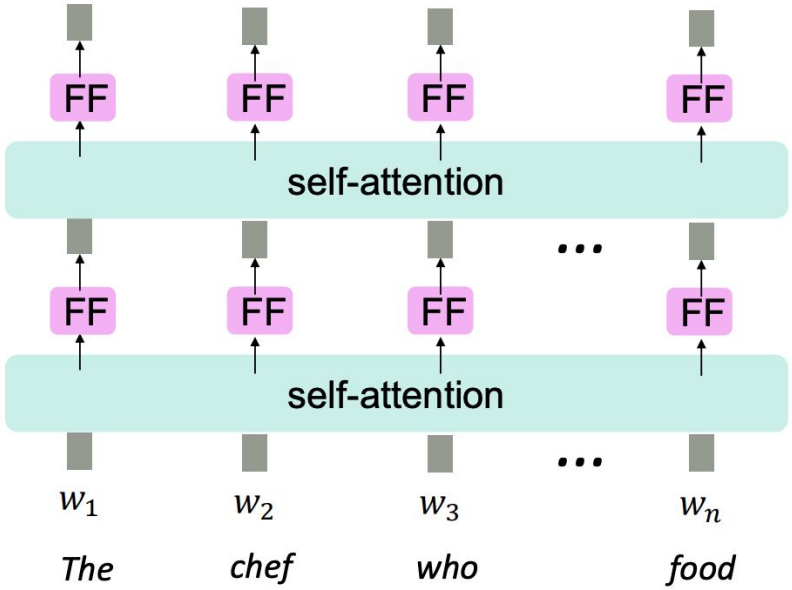
- **Problem:** there are no element-wise nonlinearities in self-attention; stacking more self-attention layers just re-averages value vectors
- **Solution:** add a **feed-forward network** to post-process each output vector.



Adding Nonlinearities in Self-Attention

- **Problem:** there are no element-wise nonlinearities in self-attention; stacking more self-attention layers just re-averages value vectors
- **Solution:** add a **feed-forward network** to post-process each output vector.





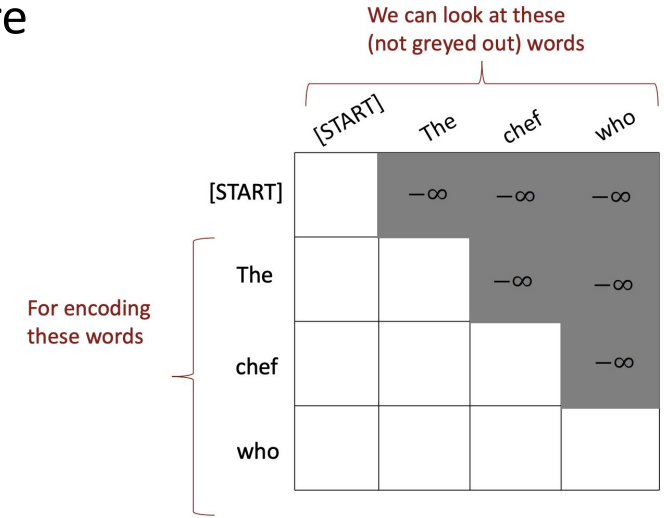
Intuition: the FF network processes the result of attention

Masking the Future in Self-Attention

- **Problem:** To use self-attention in **decoders**,
  we need to ensure we can't peek at the future
  when predicting a sequence during training
  - E.g. target sentence in translation

Masking the Future in Self-Attention

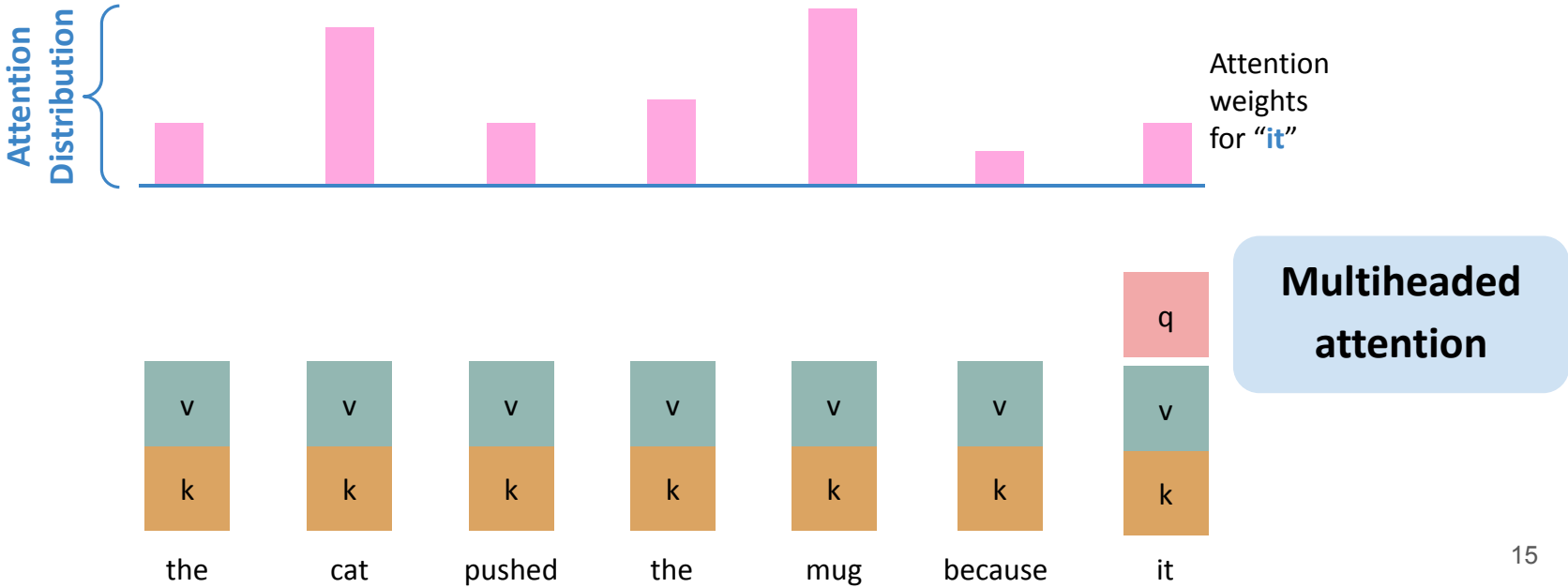
- **Problem:** To use self-attention in **decoders**, we need to ensure we can't peek at the future when predicting a sequence during training
  - E.g. target sentence in translation
- **Possible solution?** At every timestep, we could change the set of **keys and queries** to include only past words. (Inefficient!)
- **Solution:** To enable parallelization, we **mask out attention** to future words by setting attention scores to $-\infty$.



$$e_{ij}^{\text{mask}} = \begin{cases} \mathbf{q}_i \cdot \mathbf{k}_j, & j \leq i \\ -\infty, & j > i \end{cases}$$

Self-Attention and Heads

- What if we needed to pay attention to multiple different kinds of things e.g. entities, syntax
- **Solution:** Consider multiple attention computations in parallel



Transformers: Multiheaded Attention

Multiheaded Attention

- What if we want to look in multiple places in the sentence at once?
- For word i, self-attention "looks" where $(\mathbf{x}_i\mathbf{W}^Q)\cdot(\mathbf{x}_j\mathbf{W}^K)$ is high, but maybe we want to focus on different j for different reasons?
- Define **multiple attention "heads"** through multiple $\mathbf{W}^Q$,$\mathbf{W}^K$,$\mathbf{W}^V$ matrices
- Set $d_h = d/H$ (where *H* is the number of attention heads) and use per-head learned projection matrices: $\mathbf{W}_\ell^Q, \mathbf{W}_\ell^K, \mathbf{W}_\ell^V \in \mathbb{R}^{d\times d_h} \quad \ell = 1, \ldots, H$
- Each attention head performs attention independently:

$$\text{head}_\ell = \text{softmax}_{\text{row}}\left(\frac{\mathbf{Q}_\ell\mathbf{K}_\ell^\top}{\sqrt{d_h}}\right)\mathbf{V}_\ell$$

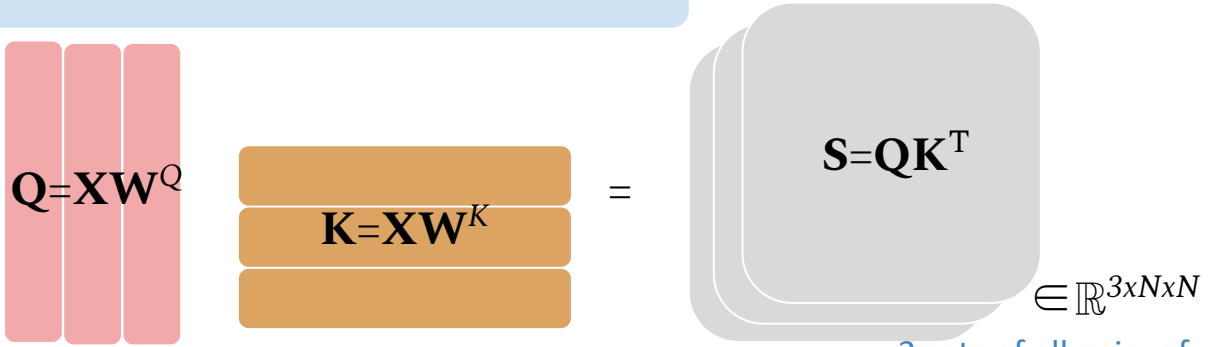
- Then the outputs of all the heads are concatenated and projected back to the token-representation dimension:

$$\text{MultiHead}(\mathbf{X}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)\mathbf{W}^O, \qquad \mathbf{W}^O \in \mathbb{R}^{Hd_h\times d}$$

Multihead Self-Attention: Visualization

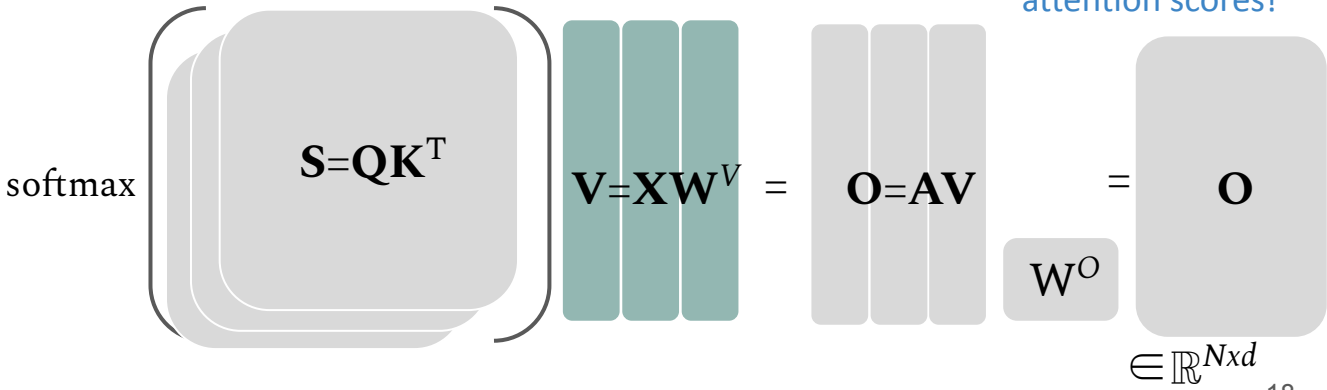
Still efficient: parallelizable

First, take the query-key dot products in one matrix multiplication



3 sets of all pairs of attention scores!

Next, softmax, and compute the weighted average with the values (with another matrix multiplication)



Scaled Dot Product Attention

- **“Scaled Dot Product”** attention aids in training.
- When dimensionality d_k becomes large, dot products between vectors tend to become large.
  - Because of this, inputs to the softmax function can be large, making the gradients small.
- We divide the attention scores by $\sqrt{d_h}$, to stop the scores from becoming large just as a function of d_k/H (the dimensionality divided by the number of heads.)

Multihead Self-Attention: Let's Revisit Linguistic Structure!

Linguistic Structure - Syntax



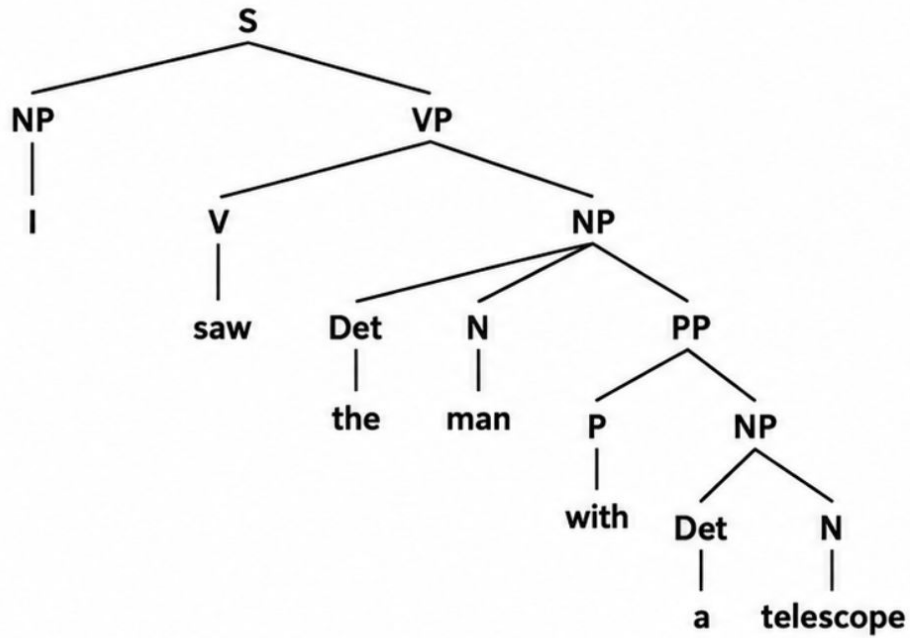
I saw the man with a telescope.

Who has the telescope?

Linguistic Structure - Syntax: PP Attachment Ambiguity

Interpretation 1: prepositional phrase modifies the noun phrase.

I saw the man with a telescope.



Bracketed Structure:

saw [the man [with the telescope]]

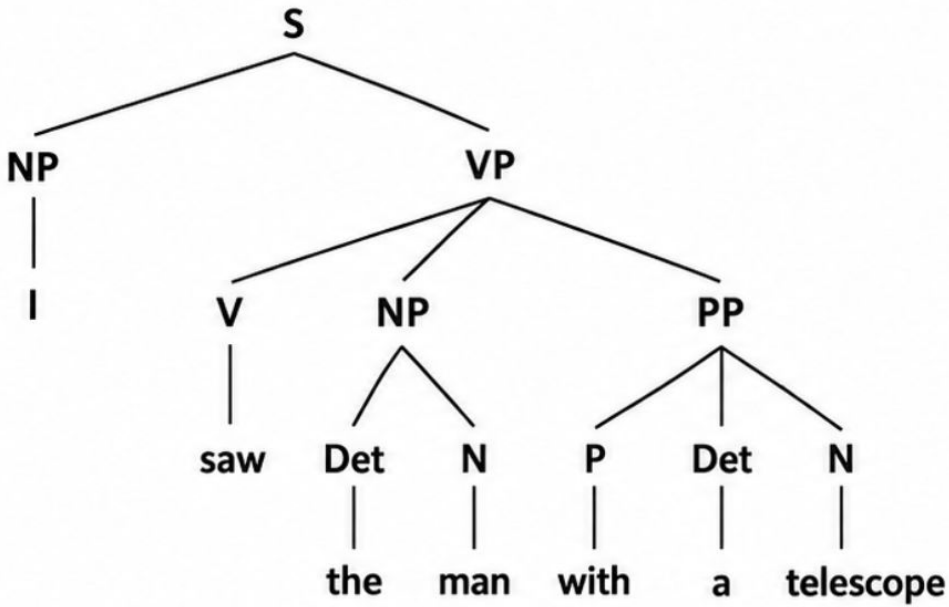
Meaning:

I saw the man who had the telescope.

Linguistic Structure - Syntax: PP Attachment Ambiguity

Interpretation 2: prepositional phrase modifies the verb phrase.

I saw the man with a telescope.



Bracketed Structure:

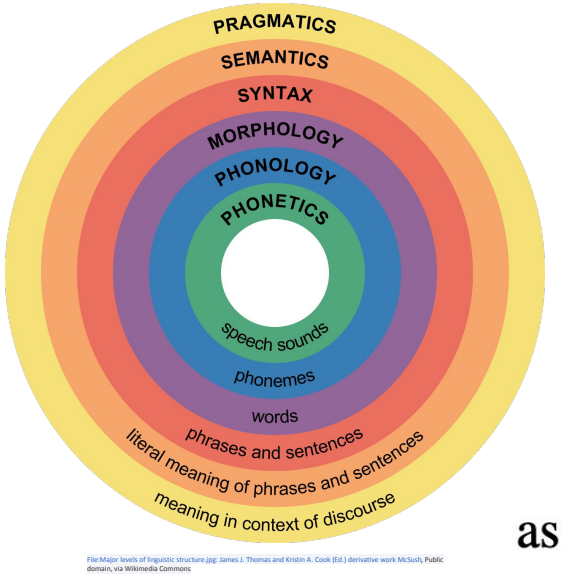
[saw the man] [with the telescope]

Meaning:

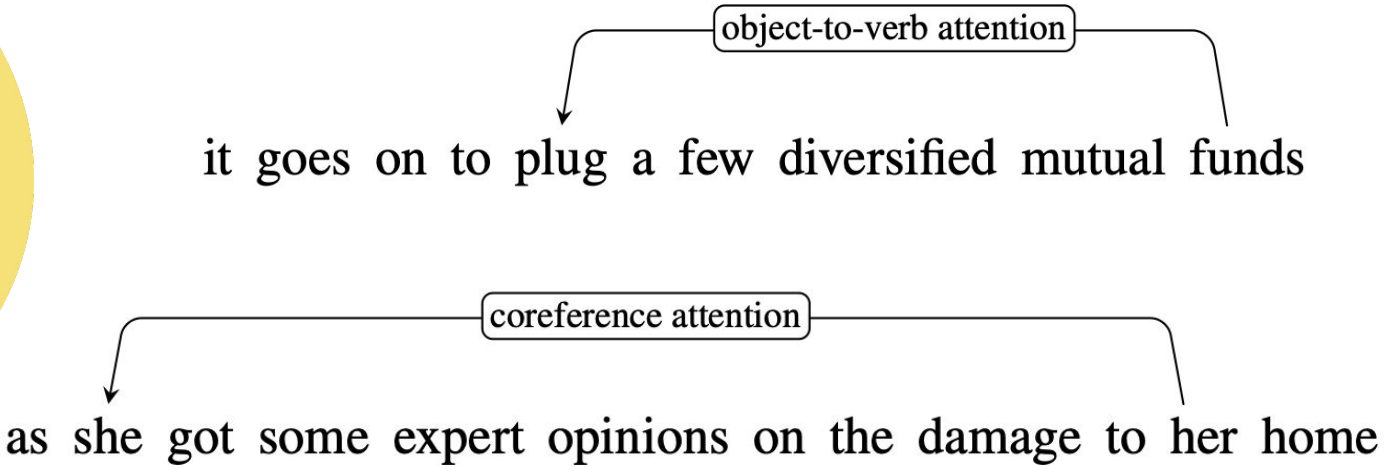
I used the telescope to see the man.

Multihead Self-Attention: Linguistic Relations & Interpretability

Goal of interpretability: understand in terms of how a neural net processes language



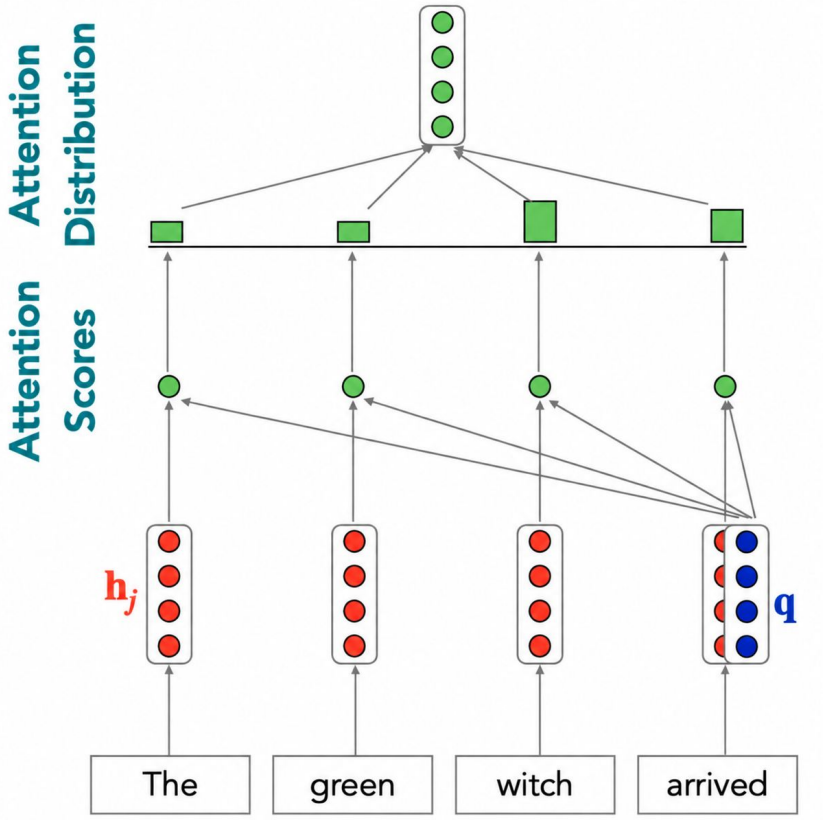
File:Major levels of linguistic structure.jpg: James J. Thomas and Kristin A. Cook (Ed.) derivative work McSush, Public domain, via Wikimedia Commons



Transformers: Positional Embeddings

Self-Attention: Order Information?

- Self-attention networks are not typically based on Recurrent Neural Nets
  - No more order information!
- Since self-attention doesn't build in order information, we need to encode the order of the sentence in our keys, queries, and values.



Missing: Order Information

- Consider representing each sequence index as a vector
  - $\mathbf{p}_i \in \mathbb{R}^d$, for $i \in \{1, 2, \ldots, n\}$ are position vectors

- Easy to incorporate this info: just add the $\mathbf{p}_i$ to our inputs!
- Recall that $\mathbf{x}_i$ is the embedding of the word at index i. The positioned embedding is:

$$\tilde{\mathbf{x}}_i = \mathbf{x}_i + \mathbf{p}_i$$

- In deep self-attention networks, typical to do this at the first layer! You could concatenate them as well, but people mostly just add them.

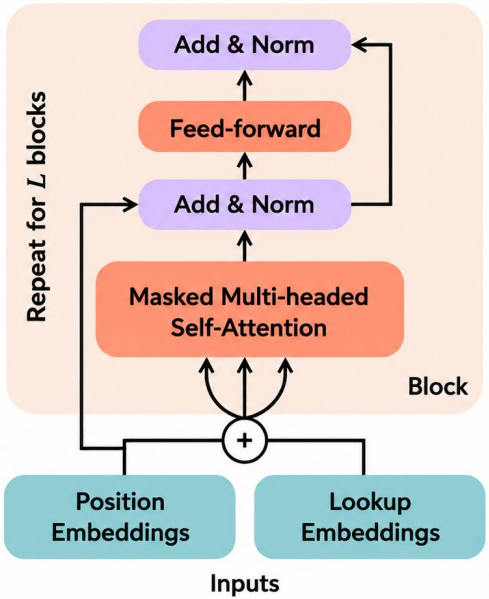
Positional Embeddings

- Maps integer inputs (for positions) to real-valued vectors
  - one per position in the entire context
- Can be randomly initialized and can let all $\mathbf{p}_i$ be learnable parameters (most common)
- Pros:
  - Flexibility: each position gets to be learned to fit the data
- Cons:
  - Definitely can't extrapolate to indices outside $1, \ldots, n$, where n is the maximum length of the sequence allowed under the architecture
  - There will be plenty of training examples for the initial positions in our inputs and correspondingly fewer at the outer length limits

Putting It All Together: Transformer Blocks

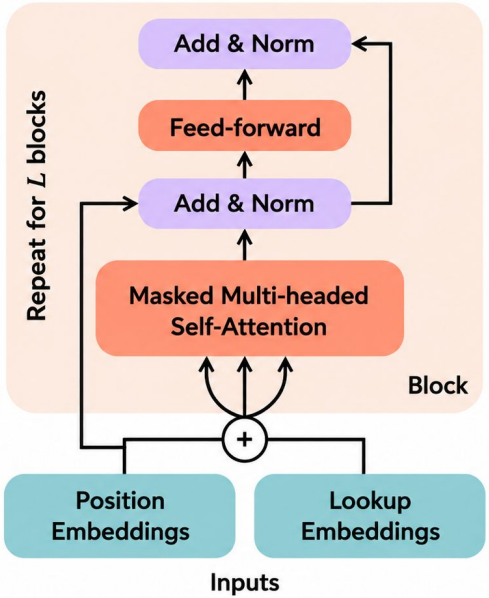
Self-Attention Transformer Building Blocks

- Self-attention:
  - the basis of the method; with multiple heads
- Position representations:
  - Specify the sequence order, since self-attention is an unordered function of its inputs.
- Nonlinearities:
  - At the output of the self-attention block
  - Frequently implemented as a simple feedforward network.
- Masking:
  - In order to parallelize operations while not looking at the future.
  - Keeps information about the future from "leaking" to the past.



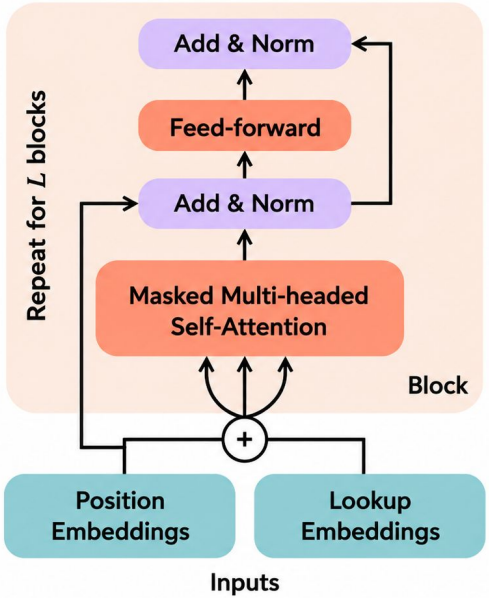
The Transformer Model

- Transformers are made up of stacks of transformer blocks, each of which is a multilayer network made by combining feedforward networks and **self-attention layers**, the key innovation of self-attention transformers
- The Transformer Decoder-only model corresponds to
  - a Transformer language model
- Lookup embeddings for tokens are usually randomly initialized



The Transformer Model

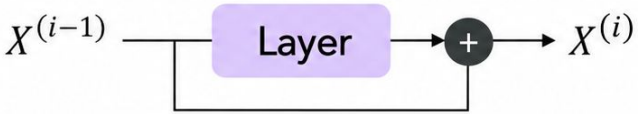
- Now that we've replaced self-attention with multi-head self-attention, we'll go through two **optimization tricks** that end up being :
  - **Residual Connections**
  - **Layer Normalization**
- In most Transformer diagrams, these are often written together as "Add & Norm"



Residual Connections



- Original Connections: $X^{(i)} = \text{Layer}(X^{(i-1)})$ where i represents the layer
- **Residual Connections**: trick to help models train better.
  - We let $X^{(i)} = X^{(i-1)} + \text{Layer}(X^{(i-1)})$
    - so we only have to learn "the residual" from the previous layer



- Allowing information to skip a layer improves learning and gives higher level layers **direct access to information** from lower layers (He et al., 2016).

Layer Normalization

- Layer normalization is another trick to help models train faster
- Idea: cut down on uninformative variation in hidden vector values by normalizing to zero mean and unit standard deviation **within each layer**
- Let $x \in \mathbb{R}^d$ be an individual (word) vector in the model.

Mean

$$\mu = \frac{1}{d}\sum_{j=1}^{d} x_j; \quad \mu \in \mathbb{R}$$

Standard deviation

$$\sigma = \sqrt{\frac{1}{d}\sum_{j=1}^{d}(x_j - \mu)^2}; \quad \sigma \in \mathbb{R}$$

Result: New vector with zero mean and a standard deviation of one.

$$\hat{x} = \frac{x - \mu}{\sigma}$$

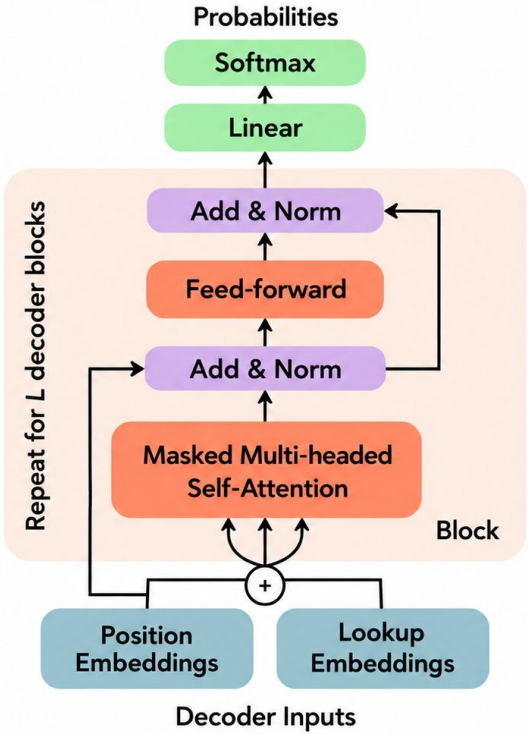
Component-wise subtraction

- Let $\gamma \in \mathbb{R}$ and $\beta \in \mathbb{R}^d$ be learned scale and shift parameters. (Can omit!)

$$\textbf{LayerNorm} = \gamma\hat{x} + \beta$$

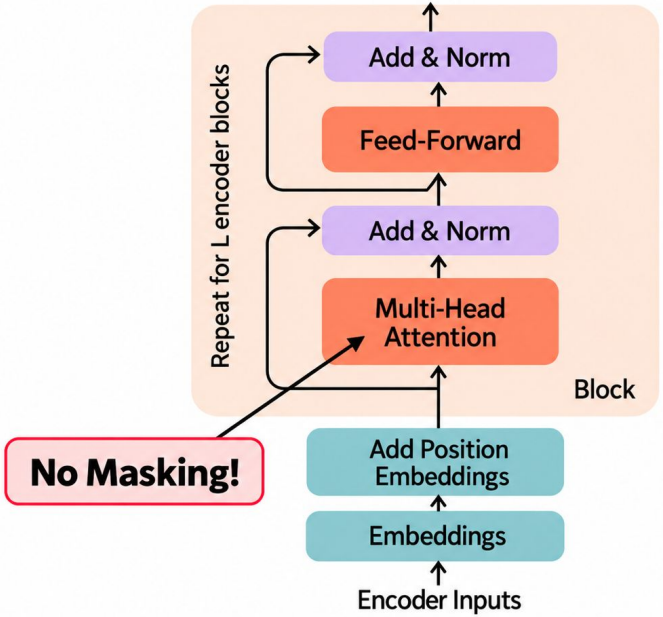
The Transformer Decoder

- The Transformer Decoder is a stack of Transformer Decoder **Blocks**.
- Each Block consists of:
  - Self-attention
  - Add & Norm
  - Feed-Forward
  - Add & Norm
- Output layer is as always a softmax layer
  - Sometimes called an **unembedding** layer



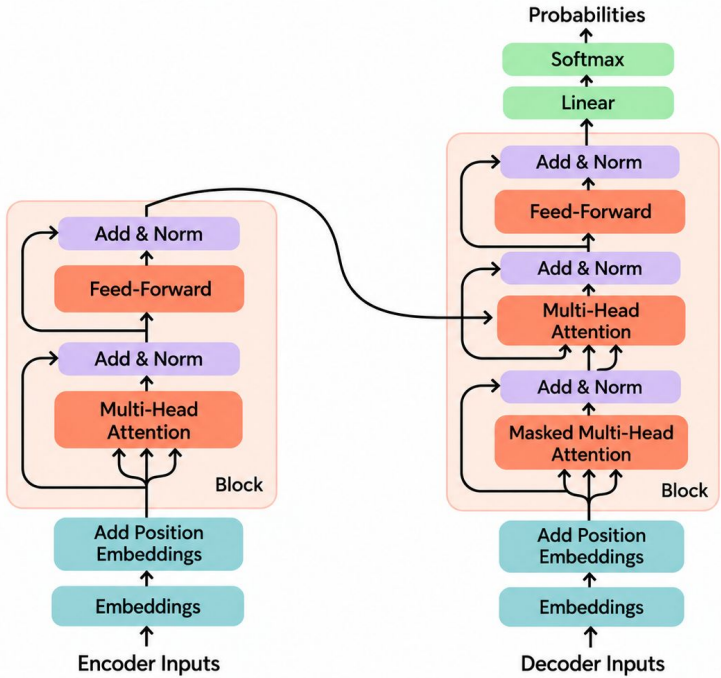
The Transformer Encoder

- The Transformer Decoder constrains to **unidirectional context**, as for **language models**.
- What if we want **bidirectional context**, like in a bidirectional RNN?
- This is the Transformer Encoder. The only difference is that we **remove the masking** in the self-attention.



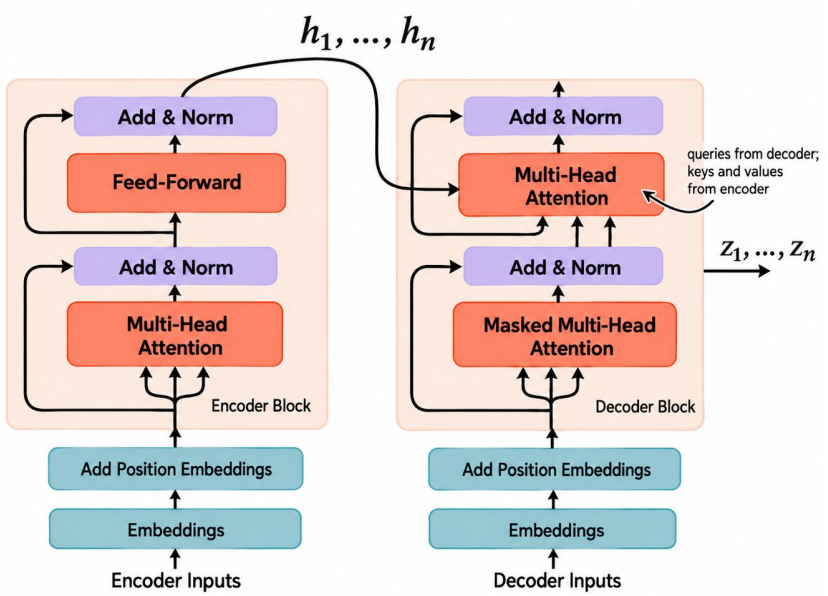
The Transformer Encoder-Decoder

- In seq2seq, we can process the source sentence with a **bidirectional** model and generate the target with a **unidirectional model**.
- For this kind of seq2seq format, we often use a Transformer Encoder-Decoder.
- We use a normal Transformer Encoder.
- Our Transformer Decoder is modified to perform **cross- attention** to the output of the Encoder.



Cross-Attention (details)

- We saw that self-attention is when keys, queries, and values come from the same source
- Let $\mathbf{h}_{1:M}$ be **output** vectors **from** the Transformer **encoder** $\mathbf{h}_j \in \mathbb{R}^{1\times d}$
- Let $z_{1:N}$ be input vectors from the Transformer **decoder** $\mathbf{z}_i \in \mathbb{R}^{1\times d}$
- Then keys and values are drawn from the encoder (like a memory)
  - $\mathbf{k}_j = \mathbf{h}_j \mathbf{W}^K$, $\mathbf{v}_j = \mathbf{h}_j \mathbf{W}^V$
- And the queries are drawn from the decoder, $\mathbf{q}_i = \mathbf{z}_i \mathbf{W}^Q$



Great Results with Transformers

First, Machine Translation from the original Transformers paper!

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

[Test sets: WMT 2014 English-German and English-French] [Vaswani et al., 2017]

Great Results with Transformers

Document generation!

Model	Test perplexity	ROUGE-L
seq2seq-attention, $L = 500$	5.04952	12.7
Transformer-ED, $L = 500$	2.46645	34.2
Transformer-D, $L = 4000$	2.22216	33.6
Transformer-DMCA, no MoE-layer, $L = 11000$	2.05159	36.2
Transformer-DMCA, MoE-128, $L = 11000$	1.92871	37.9
Transformer-DMCA, MoE-256, $L = 7500$	1.90325	38.8

[Liu et al., 2018]; WikiSum dataset

Great Results with Transformers

Document generation!

Model	Test perplexity	ROUGE-L
seq2seq-attention, $L = 500$	5.04952	12.7
Transformer-ED, $L = 500$	2.46645	34.2
Transformer-D, $L = 4000$	2.22216	33.6
Transformer-DMCA, no MoE-layer, $L = 11000$	2.05159	36.2
Transformer-DMCA, MoE-128, $L = 11000$	1.92871	37.9
Transformer-DMCA, MoE-256, $L = 7500$	1.90325	38.8

[Liu et al., 2018]; WikiSum dataset

- Before long, most Transformers results also included **pretraining**, a method we'll go over next class.
- Transformers' parallelizability allows for efficient pretraining, and have made them the de-facto standard.

TODOs:

- **Homework 2 Due: 10/23**

Another Resource: Illustrated Transformer

Next Class: Transformer LMs and Pretraining